B2. Código Ensamblador. Optimizaciones

Objetivos

Entender el papel del **programador**, del **compilador** y del **repertorio de instrucciones** del procesador (ISA) en la transformación del código fuente en lenguaje C/C++ a **instrucciones máquina**.
Reconocer las diferencias entre compiladores y el efecto de usar diferentes niveles de optimización (-O1, -O2, -O3, -Ofast ...).
Reconocer el código ensamblador del repertorio x86-64 (formato Linux).
Aprender algunas estrategias básicas de optimización del código: *loop unroll*, *code motion*, substituir funciones en línea (*function inlining*), *loop peeling* y *loop interchange*

Entender los problemas del compilador con los saltos condicionales, las llamadas a funciones y el *memory aliasing* (sinónimos).

Problema 1. Vector SUM

A continuación, se muestra el resultado de compilar un programa C++, que suma los N elementos de un vector, a un repertorio de **instrucciones máquina** x86-64, con el compilador **gcc** versión 11.2, y con la **opción de compilación** "–O2", que indica que se utilicen estrategias simples de optimización del código. Se utiliza la aplicación web "*Compiler Explorer*" (https://godbolt.org/).

```
C++
1  double f (double X[], int N)
2  {
3    int i=0;
4    double sum=0.0;
5    for (; i<N; i++)
6      sum += X[i];
7    return sum;
8  }
```

```
5          pxor    %xmm0, %xmm0
6          leaq    8(%rdi,%rax,8), %rax
7  .L3:
8          addsd   (%rdi), %xmm0
9          addq    $8, %rdi
10         cmpq    %rax, %rdi
11         jne     .L3
12         ret
13 .L4:
```

Explicar el Código Ensamblador a partir del Código Fuente: Código C/C++ equivalente

Abajo a la izquierda se muestran las 4 instrucciones máquina (ensamblador) que forman el bucle interno del programa, que es la parte del código importante respecto al rendimiento. Las instrucciones máquina de fuera del bucle, y que no se muestran, se ejecutan una vez, mientras que las de dentro del bucle se ejecutan N veces: cuanto mayor sea el valor de N mayor será el peso relativo del bucle en el rendimiento (tal como indica la Ley de Amdhal). Abajo a la derecha se muestra una representación abstracta de las 4 instrucciones máquina que usa la sintaxis del lenguaje C.

```
7  .L3:
8          addsd   (%rdi), %xmm0
9          addq    $8, %rdi
10         cmpq    %rax, %rdi
11         jne     .L3
```

Abstract ASM description

```
Register %xmm0 is sum
Register %rdi holds address X
Register %rax holds address ptrEnd=X+N

sum = sum + X[0]
X   = X + 1      // C syntax: pointer addition
c   = X != ptrEnd
if c branch to beginning
```

La primera instrucción máquina del bucle (línea 8) usa el contenido del registro **%rdi** como dirección para leer un dato de memoria. El dato leído de memoria se suma al contenido del registro **%xmm0** y se almacena en ese mismo registro destino, **%xmm0**. Esta instrucción es la que toma el papel de la operación **sum = sum + X[i]**, que es la operación básica del algoritmo **vector_SUM**. El registro **%xmm0** corresponde a la variable **sum**, y en cada iteración del bucle se le va sumando el valor de un elemento consecutivo del array.

El papel del registro **%rdi** es una combinación de las variables X e **i**. El registro **%rdi** es un apuntador, una dirección de acceso a memoria, que comienza "apuntando" al primer elemento del array X, y se va actualizando cada iteración del bucle para que apunte al siguiente elemento del vector. La segunda instrucción máquina (línea 9) suma la constante 8 al registro **%rdi**, que corresponde al tamaño de los datos del vector X[],de tipo **double**, que son 8 Bytes, **sizeof**(double) = 8. La tercera instrucción máquina (línea 10) compara el registro **%rax** con el registro **%rdi**, y luego la cuarta y última instrucción del bucle (línea 11) salta de forma condicional al principio del bucle si los dos registros comparados no tienen el mismo valor.

En lenguaje C/C++, sumar el valor 1 a un apuntador hace que "apunte" al siguiente elemento. Como el lenguaje permite al compilador "conocer" el tipo de datos al que "apunta" una dirección, en este caso (**double** *), el compilador "sabe" que sumar 1 a X es equivalente a aumentar la dirección de memoria en 8 unidades. El registro **%rax** tiene el valor que indica "cuándo" debe acabar el bucle, que corresponde con la expresión "X+N", y que es la dirección del primer elemento del array más N, o, lo que

es lo mismo, la dirección justo después del último elemento del array (elemento X[N-1]).

Resumen de optimizaciones: el compilador ha **convertido** el código que accede a los datos de un vector mediante índices en una versión que accede al vector mediante un **apuntador**; en lugar de contar las iteraciones del bucle con el índice **i**, que va de 0 a N, las iteraciones del bucle se controlan con el valor de un puntero que va de X a X+N. Esto evita tener que multiplicar y sumar para calcular la dirección de acceso a memoria de cada elemento del vector.

Desenrollar el bucle (Loop Unroll)

Una programadora aplica la siguiente transformación al programa anterior, que se denomina "desenrollar el bucle" (*loop unroll*). Al lado derecho se muestra el bucle resultante en lenguaje ensamblador cuando se compila con la opción "–O1".

```
double f (double X[], int N)
{
  int i=0;
  double sum=0.0;
  for (; i<N; i+=2)
  {
    sum += X[i];
    sum += X[i+1];
  }
  return sum;
}
```

```
11  .L3:
12          addsd   (%rax), %xmm0
13          addsd   8(%rax), %xmm0
14          addq    $16, %rax
15          cmpq    %rdx, %rax
16          jne     .L3
```

1. Escribid el código en lenguaje C equivalente al código ensamblador del bucle interno del programa anterior.
2. Calculad la relación de mejora que se produce en el número total de instrucciones ejecutadas por el programa (ICount). Aunque siempre estamos interesados en la mejora (*speedup*) del tiempo de ejecución, en este caso no tenemos información para calcular el tiempo de ejecución y nos limitamos a analizar la variación de ICount.
3. Identificad el problema funcional que puede ocurrir en el programa anterior para ciertos valores de N y proponed una modificación del código para arreglar este problema, sin perder la ventaja del "*loop unroll*"
4. Se compila el mismo código con el bucle "*desenrollado*" pero con las opciones "**–O1 –fschedule-insns**" y se obtiene el siguiente código ensamblador. Se obtiene exactamente el mismo resultado si se compila con la opción "–O2". Explicad las diferencias que se producen en el código ensamblador y proponed una hipótesis sobre cómo estos cambios pueden ayudar a que el programa se ejecute más rápido.

```
11  .L3:
12          addsd   (%rax), %xmm0
13          addq    $16, %rax
14          addsd   -8(%rax), %xmm0
15          cmpq    %rdx, %rax
16          jne     .L3
```

Problema 2. Lectura y escritura a varios arrays

El siguiente programa aplica un filtro F[] de K datos reales de precisión simple a un vector de entrada X[] de N+K-1 elementos (datos reales tipo **float**) para generar un vector de salida O[] de N elementos (**float**). Se compila el programa a un repertorio x86-64, con el compilador **gcc** v11.2, y la opción "–O1".

```
void F ( float X[], float O[], float F[], int N, int K )
{
  for (int i=0; i<N; i++)
    for (int k=K-1; k>=0; k--)
      O[i] += F[k]*X[i+k];
}
```

El programa anterior es muy compacto, aprovechando que en lenguaje C, cuando el bloque tiene una única sentencia, no se necesita usar "{" }" para marcar un bloque de sentencias. Pero es una <u>mala praxis</u> de programación, porque es fácil olvidar que al añadir más sentencias al cuerpo del bucle se necesita incluir las "llaves". Así que es buena idea incluir siempre las "llaves", tal como se muestra en la siguiente captura de pantalla, que incluye el código C y el código ensamblador resultante que corresponde con el bucle más interno de los dos (el que itera con la variable **k**).

```
void F ( float X[], float O[], float F[], int N, int K )
{
  for (int i=0; i<N; i++)
  {
    for (int k=K-1; k>=0; k--)
    {
      O[i] += F[k]*X[i+k];
    }
  }
}
```

x86-64 gcc 11.2 -O1

```
10  .L4:
11          movss   (%rdx,%rax,4), %xmm0
12          mulss   (%rdi,%rax,4), %xmm0
13          addss   (%rsi), %xmm0
14          movss   %xmm0, (%rsi)
15          subq    $1, %rax
16          testl   %eax, %eax
17          jns     .L4
```

Nota 1: hacer que el bucle interno cuente de K-1 hasta 0 (incluido), en lugar de contar desde 0 hasta K (no incluido), no hace al programa más rápido. Solamente queremos analizar esta forma de bucle para entender mejor cómo se genera el código ensamblador.

Nota 2: las instrucciones máquina de las líneas 11 y 12 (movss y mulss) realizan operaciones de lectura a memoria (denominadas LOAD, o de carga de memoria) usando un modo de direccionamiento escalado con el registro %rax. (Revisad la documentación sobre *modos de direccionamiento a memoria*).

1. Escribid el código en lenguaje C equivalente al código ensamblador del bucle interno del programa anterior. Indicad específicamente con qué variable del código C se corresponde cada uno de los siguientes registros: %rax, %rdx, %rdi y %rsi.

2. Hay dos operaciones de bajo nivel que son innecesarias dentro del bucle interno en ensamblador. Como el compilador no es capaz de generar automáticamente un código que evite la ejecución de estas operaciones innecesarias, proponed un cambio en el programa fuente (lenguaje C) para que se genere el siguiente código ensamblador, que tiene una instrucción menos (6 en lugar de 7), pero que en realidad se "ahorra" dos operaciones máquina en cada iteración del bucle.

x86-64 gcc 11.2 -O1

```
13  .L4:
14          movss   (%rdx,%rax,4), %xmm0
15          mulss   (%rdi,%rax,4), %xmm0
16          addss   %xmm0, %xmm1
17          subq    $1, %rax
18          testl   %eax, %eax
19          jns     .L4
```

Problema 3

Se compila el siguiente código en lenguaje C con el compilador gcc 11.2, opción –O3, para un repertorio x86-64. Abajo se muestran los resultados obtenidos usando *Compiler Explorer* con los parámetros adecuados. El bucle interno del programa, el que nos interesa (Ley de Amdhal), consta de 11 instrucciones máquina.

```
void f( float V1[], float V2[], float A, int N) {
  for (int i=0; i<N; i++)
    for (int x=1; x <= N; x++)
      V2[x]= A*V1[i] + 2.0*V2[x-1];
}
```

COMPILER EXPLORER

```
1  void f( float V1[], float V2[], float A, int N)
2  {
3    for (int i=0; i<N; i++)
4      for (int x=1; x <= N; x++)
5        V2[x]= A*V1[i] + 2.0*V2[x-1];
6  }
```

```
13  .L4:
14          movss   (%rdi), %xmm2
15          cvtss2sd        %xmm1, %xmm1
16          addsd   %xmm1, %xmm1
17          addq    $4, %rax
18          mulss   %xmm0, %xmm2
19          cvtss2sd        %xmm2, %xmm2
20          addsd   %xmm2, %xmm1
21          cvtsd2ss        %xmm1, %xmm1
22          movss   %xmm1, -4(%rax)
23          cmpq    %rax, %rdx
24          jne     .L4
```

1. Completar el siguiente código en lenguaje C, en el que cada instrucción que comienza con un número de línea L es directamente equivalente a la instrucción del código ensamblador original que está en la misma línea L. Ayuda: podéis usar la web *Compiler Explorer* para comprobar que vuestra propuesta de solución es correcta.

```
          float  t1f    = V2[0]
          float *ptrV2  = V2+1
          float *ptrV1  = ________
          float *ptrEnd = V2+N+1
14.         float t2f  = ____________________
15.         double t1d = (double) ___________
16.         t1d        = t1d + t1d
17.         ptrV2      = ptrV2 + 1
18.         t2f        = t2f * _______________
19.         double t2d = ____________________
20.         t1d        = t1d + t2d
21.         t1f        = ____________________
22.         ___________= t1f
23.         cond       = ptrV2 != ptrEnd
24.         if ( __________ ) goto 14
```

2. Identificar y explicar muy brevemente (20-40 palabras) qué transformación ha hecho el compilador al generar la línea 16 del código ensamblador y por qué esta transformación es susceptible de generar un código más rápido.

El compilador ha asignado a registros las variables x, N, V1, V2 y A del programa, para que los accesos a los datos que contienen sean mucho más rápidos. En realidad, la variable x no se usa en el código ensamblador, sino que se ha transformado el uso de un contador del bucle en el uso de apuntadores a memoria (ptrV1 y ptrV2). Pero los accesos a los elementos de los vectores V1 y V2, no se pueden substituir por accesos a registros, y tienen que hacerse a memoria, usando operaciones a nivel de máquina denominadas LOAD y STORE.

3. Identifica qué operaciones LOAD y/o STORE ha conseguido evitar el compilador en cada iteración del bucle más interno y explica por qué. (20-100 palabras)

4. Explicar con 20-100 palabras (vuestras propias palabras) qué hacen las instrucciones máquina de las líneas 15, 19 y 21, y por qué las genera el compilador. Podéis ver: https://www.felixcloutier.com/x86/cvtss2sd.

Se modifica el programa anterior simplemente añadiendo a la constante numérica 2.0 el sufijo "f", es decir, 2.0f. El resultado de compilar con gcc 11.2 opción –O3, es un bucle interno de 8 instrucciones máquina. Lo tenéis que obtener vosotros mismos usando la web *Compiler Explorer*.

5. Explicar con 10-30 palabras por qué se ha reducido el número total de instrucciones del programa.

6. Todavía es posible eliminar dos instrucciones máquina dentro del bucle más interno del programa. Propón los cambios mínimos necesarios en el código C (además de usar la constante 2.0f) y explica muy brevemente qué operaciones se evitan (seguid compilando con gcc 11.2, opción –O3).

Problema 4. Estructuras de Datos (struct)

Se diseña un programa que accede a una estructura de datos de C (*struct*) que contiene cuatro campos (*fields*), cada uno de un tipo diferente. El programa se compila con gcc 9.2.0 –O1 y produce el bucle de la derecha, con 7 instrucciones.

```c
struct SS { int I; float F; double D; char C; };
double F (struct SS V[], int N) {
  double SD=0.0;
  for (int i=0; i<N; i++)
    SD += V[i].I + V[i].D;
  return SD;
}
```

```
 9  .L3:
10          pxor    %xmm0, %xmm0
11          cvtsi2sdl       (%rax), %xmm0
12          addsd   8(%rax), %xmm0
13          addsd   %xmm0, %xmm1
14          addq    $24, %rax
15          cmpq    %rdx, %rax
16          jne     .L3
```

Se propone la siguiente modificación al programa y se obtiene el código ensamblador de abajo a la derecha (gcc 9.2.0 –O1).

```c
struct SS { int I; float F; double D; char C; };
double F (struct SS V[], int N) {
  long   SI=0;  double SD=0.0;
  for (int i=0; i<N; i++) {
    SI += V[i].I;  SD += V[i].D;
  }
  return SI+SD;
}
```

```
10  .L3:
11          movslq  (%rax), %rcx
12          addq    %rcx, %rdx
13          addsd   8(%rax), %xmm1
14          addq    $24, %rax
15          cmpq    %rsi, %rax
16          jne     .L3
```

1. La nueva versión del programa debería ejecutarse en menos tiempo, pero puede tener un problema si el valor de la variable SI del nuevo programa se "desborda" al sumar muchos valores muy grandes (si se produce *overflow*). ¿A partir de qué valor aproximado de N se podría correr el riesgo de que la suma **`SI += V[i].I;`** pueda producir *overflow* con un número positivo muy grande?

2. La primera instrucción del bucle interno del código ensamblador del nuevo programa tiene el código mnemónico **movslq**, cuya descripción es la siguiente. Explicar muy, muy brevemente el papel de esta instrucción en el nuevo programa.

> MOVSLQ is move and sign-extend a value from a 32-bit source to a 64-bit destination.

3. ¿Cuántos Bytes ocupa cada elemento del vector V[]? En otras palabras, ¿cuánto ocupa la estructura de datos SS?

4. Indicar cuánto espacio ocupa en memoria una variable declarada del tipo descrito por la siguiente estructura:

```
struct SS { float F; double D; char C1; int I; char C2; };
```

Problema 5. Valor Absoluto de la resta

Se codifica el problema de calcular las diferencias absolutas entre los valores de dos vectores A y B, ambos de tamaño N. El resultado es otro vector de N elementos, y cada elemento se calcula como el valor absoluto de las diferencias entre los elementos correspondientes de los vectores A y B. Además, se calcula la máxima diferencia de todas, y se devuelve este valor como resultado de la función.

$A = \{ a_0, a_1, a_2, \dots a_{N-1} \}$,
$B = \{ b_0, b_1, b_2, \dots b_{N-1} \}$
$\Rightarrow$
$C = \{ c_0, c_1, c_2, \dots c_{N-1} \}$: $c_i = |a_i - b_i|$

A la derecha se muestra el código C y abajo se muestra el código ensamblador que corresponde con el bucle **for** interno que resulta al compilar con **gcc** v12.2.0, opción –O3, para ISAs x86-64 y rv64gc.

```
double AbsDiff (double A[], double B[], double C[], int N) {
  double Max=0;
  for (int i=0; i<N; i++) {
    if (A[i] < B[i])
      C[i] = B[i]-A[i];
    else
      C[i] = A[i]-B[i];
    if (Max < C[i])
      Max = C[i];
  }
  return Max;
}
```

x86-64 gcc 12.2 | -O3 -fopt-info

```
.L14:
        subsd   %xmm2, %xmm1
        movapd  %xmm1, %xmm0
.L5:
        movsd   %xmm0, (%rdx,%rax)
        maxsd   %xmm3, %xmm0
        addq    $8, %rax
        movapd  %xmm0, %xmm3
        cmpq    %rax, %rcx
        je      .L1
.L7:
        movsd   (%rdi,%rax), %xmm2
        movsd   (%rsi,%rax), %xmm1
        comisd  %xmm2, %xmm1
        ja      .L14
        subsd   %xmm1, %xmm2
        movapd  %xmm2, %xmm0
        jmp     .L5
```

A B C D

Output (0/1) x86-64 gcc 12.2 i - 623ms (7803B) ~473 lines filtered

Output of x86-64 gcc 12.2 (Compiler #1)

Wrap lines | Select all

```
<source>:5:12: optimized: sinking common stores to *_36
Compiler returned: 0
```

RISC-V rv64gc gcc 12.2.0 | -O3 -fopt-info

```
AbsDiff:
        fmv.d.x fa0,zero
        ble     a3,zero,.L8
        slli    a3,a3,3
        add     a4,a0,a3
.L7:
        fld     fa5,0(a0)
        fld     fa4,0(a1)
        flt.d   a5,fa5,fa4
        beq     a5,zero,.L12
        fsub.d  fa5,fa4,fa5
.L5:
        flt.d   a5,fa0,fa5
        fsd     fa5,0(a2)
        addi    a0,a0,8
        addi    a1,a1,8
        addi    a2,a2,8
        beq     a5,zero,.L6
        fmv.d   fa0,fa5
.L6:
        bne     a0,a4,.L7
        ret
.L12:
        fsub.d  fa5,fa5,fa4
        j       .L5
```

A B C D E F

Output (0/1) RISC-V rv64gc gcc 12.2.0 i - 519ms (6603B) ~468 lines filte

Output of RISC-V rv64gc gcc 12.2.0 (Compiler #1)

Wrap lines | Select all

```
<source>:5:12: optimized: sinking common stores to *_36
Compiler returned: 0
```

Las sentencias if-else y if del programa C necesitan diferentes "caminos" en la ejecución, lo que supone tener que utilizar saltos condicionales hacia "arriba" y hacia "abajo", que complican la comprensión del código, y que también la pueden ralentizar. Es una "suerte" no tener que programar en ensamblador, y que sea el compilador el que genere este código. En las capturas anteriores se marcan los **bloques básicos** (*Basic Block* = BB) del bucle interno. Un bloque básico se define como un grupo de instrucciones consecutivas con el máximo de instrucciones tales que el único punto de entrada es la primera instrucción, a la que se salta desde al menos un lugar del programa, y el único punto de salida es la última instrucción, que en ocasiones es una instrucción de salto, pero que también puede ser la anterior al punto de entrada del BB que le sigue.

En el ISA x86-64, la instrucción **je**, al final del bloque B, es un salto condicional, con la condición de igualdad (jump if equal). La instrucción **ja**, al final del bloque C, compara si el primer valor es mayor que el segundo (jump if above). La instrucción **jmp**, al final del bloque D, es un salto incondicional (jump). Las líneas 13, 20 y 21 contienen instrucciones que acceden a memoria: *store to memory*, la primera, y *load from memory* las dos últimas. Las tres instrucciones usan un modo de direccionamiento de tipo indirecto e indexado. La dirección se calcula sumando el contenido de los dos registros que están entre paréntesis: *%rax* representa siempre el registro índice, y el otro registro representa el registro base (aunque en realidad los dos registros se usan para lo mismo, y son intercambiables). La instrucción **comisd** de la línea 22 compara los valores de los registros %xmm2 y %xmm1 junto al salto condicional **ja** de la línea 23. Es complicado verificar qué registro tiene que tener un valor más alto para que el salto se produzca. Según el caso, se ejecutan las instrucciones del bloque A o las del bloque D. La instrucción **subsd** (línea 24) también es confusa, porque no es intuitivo saber qué valor se resta de qué valor. Mirando con detenimiento la explicación en la web "https://www.felixcloutier.com/x86/subsd" averiguamos que al operando de la derecha, donde se deja el resultado, se le resta el operando de la izquierda. Otro detalle muy interesante es que el compilador genera una instrucción específica, **maxsd**, para calcular el máximo entre dos números reales representados en punto-flotante (línea 14). El compilador es capaz de "deducir" que la sentencia condicional `if (Max < C[i]) Max= C[i];` es un patrón especial que se corresponde con la operación de calcular el máximo. El compilador se diseña para ser capaz de identificar ciertos casos especiales que aparecen con cierta frecuencia en los programas, y así proporcionar un código más eficiente que el que se obtendría siguiendo las reglas generales de generación y optimización de código.

En el ISA rv64gc, la instrucción **beq**, al final de los bloques A y C, es un salto condicional, con la condición de igualdad (branch if equal). La instrucción **bne**, que es la única del bloque E, es un salto condicional de desigualdad (branch if not equal). La instrucción **j**, al final del bloque F, es un salto incondicional (jump). La instrucción **ret** sirve para retornar a la función que ha llamado a esta función (return). Al revés que en el ISA x86-64, el operando destino es el de más a la izquierda. La instrucción **flt** compara si un valor en representación en punto flotante es menor que otro (float lower than). La extensión **.d** indica que los números reales son de doble precisión (64 bits = 8 Bytes).

1. Calcula el número promedio de instrucciones máquina ejecutadas en cada iteración del bucle del código x86-64, asumiendo un valor de N grande. Observa que no todos los BBs se ejecutan en cada iteración del bucle, y que el promedio de instrucciones ejecutadas estará en una horquilla entre un valor mínimo y un valor máximo. Analizando el código ensamblador previo, que no se muestra, se puede deducir que la **entrada** al bucle es el bloque básico C (etiqueta ".L7"). La **salida** del bucle es el final del bloque básico B, (salto condicional que salta a la etiqueta ".L1").

2. Haz el mismo cálculo de la pregunta anterior, pero para el código rv64gc. En este caso, la **entrada** al bucle es el bloque básico A y la **salida** es el bloque básico E.

Se compila el mismo código C con **clang** 15.0.0 y opciones -O3 y -Rpass (la opción **Rpass** indica las optimizaciones realizadas por el compilador). El resultado se muestra a la derecha y abajo. Ahora el bucle consiste en un único Bloque Básico. Sin analizar en detalle el código ensamblador, porque en breve analizaremos códigos más compactos y eficientes producidos al cambiar el programa/compilador, queremos destacar algunas ideas:

```
Output (0/7)  x86-64 clang 15.0.0  i  - 611ms (215418) ~452 lines
Output of x86-64 clang 15.0.0 (Compiler #1)
A  Wrap lines  Select all
example.c:8:15: remark: load of type double
eliminated [-Rpass=gvn]
    if (Max < C[i])
              ^
example.c:3:3: remark: unrolled loop by a factor of 2
with run-time trip count [-Rpass=loop-unroll]
  for (int i=0; i<N; i++) {
  ^
```

```
18  .LBB0_8:                                  # =>This
19          movsd   (%rdi,%rax,8), %xmm1
20          movsd   (%rsi,%rax,8), %xmm2
21          movapd  %xmm2, %xmm3
22          subsd   %xmm1, %xmm3
23          movapd  %xmm1, %xmm4
24          subsd   %xmm2, %xmm4
25          cmpltsd %xmm2, %xmm1
26          andpd   %xmm1, %xmm3
27          andnpd  %xmm4, %xmm1
28          orpd    %xmm3, %xmm1
29          movlpd  %xmm1, (%rdx,%rax,8)
30          maxsd   %xmm0, %xmm1
31          movsd   8(%rdi,%rax,8), %xmm0
32          movsd   8(%rsi,%rax,8), %xmm2
33          movapd  %xmm2, %xmm3
34          subsd   %xmm0, %xmm3
35          movapd  %xmm0, %xmm4
36          subsd   %xmm2, %xmm4
37          cmpltsd %xmm2, %xmm0
38          andpd   %xmm0, %xmm3
39          andnpd  %xmm4, %xmm0
40          orpd    %xmm3, %xmm0
41          movlpd  %xmm0, 8(%rdx,%rax,8)
42          maxsd   %xmm1, %xmm0
43          addq    $2, %rax
44          cmpq    %rax, %rcx
45          jne     .LBB0_8
```

- El bucle interno contiene las mismas instrucciones repetidas dos veces.
- Las instrucciones que leen de memoria (líneas 19, 20, 31 y 32) y las que escriben en memoria (líneas 29 y 41) usan un modo de direccionamiento de tipo indexado y escalado. La dirección se calcula multiplicando el contenido del registro %rax (el especificado más a la derecha dentro de los paréntesis) por el valor 8 (la *escala*) y el resultado de esta multiplicación se suma con el contenido del primer registro (%rdi, %rsi o %rdx).
- Las instrucciones de las líneas 31, 32 y 41, incluyen un valor de desplazamiento en su direccionamiento a memoria: el número 8 justo antes del paréntesis. Este valor se suma a la dirección de memoria calculada tal como se indica en el punto anterior.
- Se usan instrucciones que manejan el contenido de los registros como conjuntos de bits: **andpd**, **andnpd** y **orpd** realizan operaciones bit a bit con las funciones **and**, **and-not** y **or**. Estas instrucciones se usan para borrar el bit de signo del número, que es el bit más significativo de los 64 bits del formato **double**, y así obtener la diferencia positiva. Esto nos puede dar una idea para modificar el programa, que veremos a continuación.

3. Deberías saber reconocer las dos transformaciones que ha realizado el compilador **clang**, y cuyas explicaciones se muestran en la ventana "Output", como consecuencia de usar la opción -Rpass. Explica ambas transformaciones. Explica también por qué se suma el valor 2 al registro %rax en la línea 43, y por qué se usa el valor de desplazamiento 8 en las líneas 31, 32 y 41.
4. Calcula la relación de mejora (*speedup*) respecto al número total de instrucciones máquina ejecutadas de esta nueva versión del código ensamblador respecto a la primera versión, generada con el compilador **gcc**. Hazlo considerando el mejor caso posible y el peor caso posible para la primera versión. No podemos calcular la mejora en tiempo porque no estamos ejecutando el programa y no conocemos el valor de IPC que se obtendrá al ejecutarlo en un procesador real.

Se propone una codificación alternativa, que se muestra abajo a la izquierda, usando la función predefinida fabs() de la biblioteca <math.h>. Se compila la nueva versión con **gcc** v12.2.0, opción –O3, para ISAs x86-64 y rv64gc. El código ensamblador resultante para el bucle interno de la función **AbsDiff2** se muestra a la derecha, para cada ISA.

```
#include <math.h>
double AbsDiff2 (double A[], double B[],
                 double C[], int N) {
  double Max=0;
  for (int i=0; i<N; i++) {
    C[i] = fabs(A[i]-B[i]);
    if (Max < C[i])
      Max = C[i];
  }
  return Max;
}
```

x86-64 gcc 12.2 | -O3 -fopt-info

```
9   .L5:
10          movsd   (%rdi,%rax), %xmm0
11          subsd   (%rsi,%rax), %xmm0
12          andpd   %xmm2, %xmm0
13          movsd   %xmm0, (%rdx,%rax)
14          maxsd   %xmm1, %xmm0
15          addq    $8, %rax
16          movapd  %xmm0, %xmm1
17          cmpq    %rax, %rcx
18          jne     .L5
```

Output (0/0) x86-64 gcc 12.2 - 642ms (79858) ~496 lines filtered

Output of x86-64 gcc 12.2 (Compiler #1)

Compiler returned: 0

RISC-V rv64gc gcc 12.2.0 | -O3 -fopt-info

```
6   .L5:
7           fld     fa5,0(a0)
8           fld     fa4,0(a1)
9           addi    a0,a0,8
10          addi    a1,a1,8
11          fsub.d  fa5,fa5,fa4
12          addi    a2,a2,8
13          fabs.d  fa5,fa5
14          flt.d   a5,fa0,fa5
15          fsd     fa5,-8(a2)
16          beq     a5,zero,.L3
17          fmv.d   fa0,fa5
18  .L3:
19          bne     a4,a0,.L5
20          ret
```

Output (0/0) RISC-V rv64gc gcc 12.2.0 - 671ms (70528)

Output of RISC-V rv64gc gcc 12.2.0 (Compiler #1)

Compiler returned: 0

5. ¿Por qué no aparece la llamada a la función **fabs**() en el código ensamblador del último programa?
6. La instrucción de la línea 16 del código x86-64, movapd, que mueve el contenido del registro de la izquierda (*%xmm0*) al registro de la derecha (*%xmm1*), se puede evitar. ¿Cómo?
7. ¿Qué hace la instrucción de la línea 15 del código x86-64, addq, y por qué se usa el valor 8?
8. Calcula la relación de mejora (*speedup*) para el número total de instrucciones máquina ejecutadas de la nueva versión del código ensamblador respecto a la primera de todas las versiones, para cada ISA.

Problema 6

El siguiente programa Cython (abajo a la izquierda) calcula una cierta expresión con los datos de dos arrays de entrada, V1 y V2, para dejar el resultado en el vector V2. Los datos son de tipo real de precisión simple y los vectores tienen N+1 elementos. El bucle interno en código ensamblador de la función f2() se muestra a la derecha.

```
 14: @cython.wraparound(False)
 15: @cython.boundscheck(False)
+16: def f2( float [::1] V1, float [::1] V2,
 17:          float A, int N):
 18:    cdef int x
+19:    for x in range(N):
+20:      V2[x+1] = 3.0*(A*V1[x+1] + 2.0*V2[x])
```

```
.L3:
  movss   (%rsi,%rax,4),%xmm2
  cvtss2sd %xmm1,%xmm1
  addsd   %xmm1,%xmm1
  mulss   %xmm0,%xmm2
  cvtss2sd %xmm2,%xmm2
  addsd   %xmm2,%xmm1
  mulsd   %xmm3,%xmm1
  cvtsd2ss %xmm1,%xmm1
  movss   %xmm1,(%rcx,%rax,4)
  add     $1,%rax
  cmp     %rax,%rdx
  jne     .L3
```

1. ¿Por qué aparecen las instrucciones cvtss2sd y cvtsd2ss y cómo podemos evitar que aparezcan?
2. ¿Por qué dentro del bucle interno del programa solamente se ejecutan 2 instrucciones máquina de multiplicar, mulss y mulsd, cuando en el programa aparecen 3 operaciones de multiplicar?
3. Indicar con qué variable del código Cython original se corresponde cada uno de los siguientes registros: %rsi, %rcx, %rax, %rdx, %xmm0 y %xmm3.
4. ¿Por qué dentro del bucle interno del programa solamente se ejecuta 1 instrucción máquina de lectura a memoria, o LOAD, cuando en el programa se indica que se deben leer datos de los arrays V1 y V2?
5. Además del cambio necesario para evitar el uso de las instrucciones cvtss2sd y cvtsd2ss, propón un cambio que reduzca significativamente el número total de instrucciones ejecutadas en el programa.

Problema 7

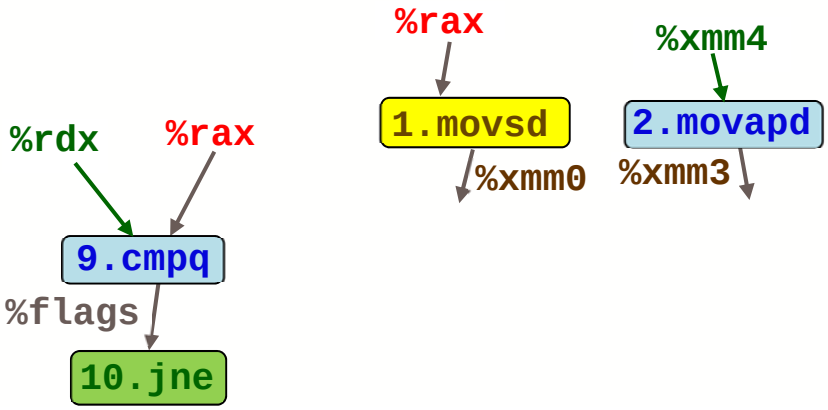
Se analiza el siguiente programa en lenguaje Cython y el código ensamblador generado al compilar (cythonize+gcc) y que corresponde al bucle FOR más interno

```
cdef void F ( double [::1]a, double [::1]b, double [::1]c, int N )
  cdef double M, T, R
  cdef int i, j, k
  for i in range(N):
    for j in range(N):
      M, T, R = c[j], 1.0, a[i]
      for k in range(N):
        T = M*a[k] + R*b[i]
        M = T if T>M else M   # C language→ M = T>M? T: M;
        R = T if T<R else R   # C language→ R = T<R? T: R;
      c[j] = T + R
```

```
.L4:
1.  movsd   (%rax), %xmm0
2.  movapd  %xmm4, %xmm3
3.  addq    $8, %rax
4.  mulsd   %xmm1, %xmm3
5.  mulsd   %xmm2, %xmm0
6.  addsd   %xmm3, %xmm0
7.  maxsd   %xmm0, %xmm2
8.  minsd   %xmm0, %xmm1
9.  cmpq    %rax, %rdx
10. jne     .L4
```

El comando perf al ejecutar el programa en un procesador real y con un tamaño de problema definido por N= 1000 indica que se obtiene un IPC de 0,8, y que la frecuencia del reloj del procesador es de 4 GHz.

1. Indicad la complejidad computacional del programa, y calculad el número total aproximado de instrucciones máquina ejecutadas (ICount) si se ejecuta la función F() una vez, y el tiempo total de la ejecución, medido en segundos. Mostrad el cálculo completo que da lugar a los valores de respuesta.
2. Generad el grafo de flujo de datos (*dataflow*) o grafo de dependencias del programa ensamblador. En este grafo, representad cada instrucción como un nodo etiquetado con el número y mnemónico de la instrucción, por ejemplo, "1.movsd" o "2.movapd", tal como se muestra en el ejemplo incompleto de abajo. Representad todas las flechas que indican datos de entrada a una instrucción y datos de salida producidos por una instrucción, y conectad todas estas flechas del grafo para reflejar con precisión qué datos de salida de las instrucciones son datos de entrada de otras instrucciones. Cada flecha se debe etiquetar con el nombre del registro que contiene el dato producido/consumido, como por ejemplo "%rax" o "%xmm4".
3. Escribid la representación abstracta de las 10 instrucciones máquina usando la sintaxis del lenguaje C. Indicad para cada registro diferente usado en el código ensamblador con qué variable del código Cython se corresponde, o, si el registro corresponde con una variable temporal no definida en el código Cython, asignadle un nombre nuevo.
4. Explicad muy brevemente qué transformaciones ha realizado el compilador respecto al código original.



Problema 8

Se analiza la siguiente función f2() en lenguaje Cython, el código ensamblador generado al compilar (cythonize+gcc) y que corresponde al bucle FOR más interno, y alguna métrica de rendimiento obtenida al ejecutar con X=100.000

```
cdef float f1( float A, float B, float L):
  return L*(2*A + B)
```

```
@cython.wraparound(False)
@cython.boundscheck(False)
cdef void f2( float [::1] A, float [::1] B, int X):
  cdef int i, j
  for i in range(X-1):
    for j in range(X):
      A[i+1] = f1(A[i], B[j], 0.5)
```

```
7b0: ┌─→pxor    %xmm0,%xmm0
     │  add     $0x4,%rax
     │  cvtss2sd -0x4(%rax),%xmm0
     │  addsd   %xmm1,%xmm0
     │  mulsd   %xmm2,%xmm0
     │  cvtsd2ss %xmm0,%xmm0
     │  movss   %xmm0,0x4(%rdx)
     ├─cmp     %rax,%rcx
     └─jne     7b0
```

```
$ perf stat python -c "import PRB1 as S; S.run(100000)"

PRUEBA1. X= 100000
CheckSum(A)= 94170.609375

     33.589.804.225    cycles

    11,873766345 seconds time elapsed
    11,846945000 seconds user
     0,024997000 seconds sys
```

1. Indica la complejidad computacional del programa, sin explicar nada más.
2. Indica el número total aproximado de instrucciones máquina ejecutadas (ICount) si se ejecuta la función f2() una vez con X=100.000. Muestra el cálculo completo que da lugar al valor final.
3. Calcula el valor del IPC y la frecuencia de reloj de la ejecución anterior. Muestra el cálculo completo.
4. Indica para cada registro diferente usado en el código ensamblador con qué variable o expresión del código Cython se corresponde. Si el registro corresponde con varios valores durante la ejecución, escribe **varios** e indica uno de estos valores.
   %rax; %rcx; %rdx; %xmm0; %xmm1; %xmm2
5. Explica muy brevemente qué transformaciones ha realizado el compilador respecto al código original.

Versión 2: una programadora modifica ligeramente el código. Al compilar se genera el siguiente código ensamblador y la ejecución obtenida con X=100.000 da lugar a las siguientes métricas de rendimiento

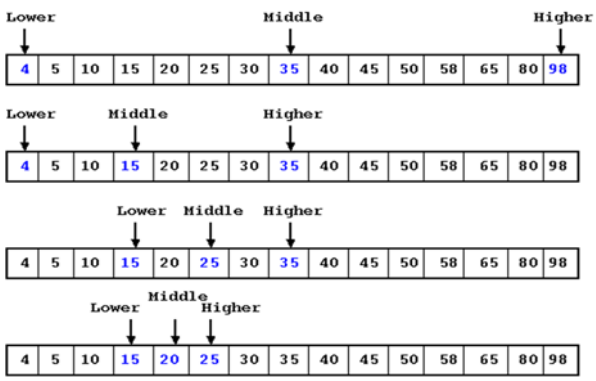
```
7a8: ┌─→movss   (%rax),%xmm0
     │  add     $0x4,%rax
     │  addss   %xmm1,%xmm0
     │  mulss   %xmm2,%xmm0
     │  movss   %xmm0,0x4(%rdx)
     ├─cmp     %rax,%rcx
     └─jne     7a8
```

```
PRUEBA1. X= 100000
CheckSum(A) =  94170.609375

    20.790.738.541      cycles

   7,365918706 seconds time elapsed
   7,325804000 seconds user
   0,038993000 seconds sys
```

6. Calcula la relación de mejora (*speedup*) que se consigue con la nueva versión del programa. Muestra el cálculo completo.
7. Explica el pequeño cambio que se ha debido hacer en el programa y arguméntalo muy brevemente.
8. Propón un cambio en el programa, además del ya realizado en la versión 2, para que se ejecute más rápido. Explica por qué el compilador no es capaz de hacer la modificación que propones. Argumenta cuál puede ser la mejora de rendimiento.

Problema 9

La siguiente función, **BinSearch**, busca M números almacenados en el vector **Val**[] dentro del vector **Idx**[], que contiene N números ordenados de menor a mayor. El resultado son las posiciones en el vector de cada uno de los M números buscados, que se almacenan en el vector **Pos**[]. Para realizar la búsqueda se aprovecha que los números están ordenados y se utiliza el algoritmo de búsqueda binaria. A la derecha se muestra un esquema visual del funcionamiento del algoritmo (bucle **while** más interno).




```
@cython.wraparound(False)
@cython.boundscheck(False)
cdef void BinSearch ( int [::1] Val, int [::1] Idx, int [::1] Pos, int N, int M ):

  cdef int i, Left, Right, Middle, Size, value, V

  for i in range(M):
    V = Val[i]
    Left =-1
    Right= N
    Size = (Right - Left) >> 1   # integer division by 2 implemented by bit shifting
    while (Size > 0):
      Middle = Left + Size
      value  = Idx[Middle]
      if (value < V):
        Left = Middle
      else:
        Right= Middle
      Size = (Right - Left) >> 1
    Pos[i] = Right
```

Abajo se muestran los resultados de rendimiento obtenidos al ejecutar en el procesador del Laboratorio el problema para buscar M= 100 millones de números en un array ordenado con N= 1 millón (izquierda), N= 2 millones (derecha) y N= 4 millones (abajo) de elementos.

```
+++++ Cython code compiled with gcc 10.2 +++++++++++++++++
$ perf stat python -c "import BS; BS.run(1000000,100000000)"

      12.507,73 msec task-clock       #  1,000 CPUs utilized
 42.234.755.924      cycles           #  3,377 GHz
 19.141.401.379      instructions     #  0,45  insn per cycle

      12,512287489 seconds time elapsed
```

```
$ perf stat python -c "import BS; BS.run(2000000,100000000)"

      17.371,92 msec task-clock       #  1,000 CPUs utilized
 58.703.732.831      cycles           #  3,379 GHz
 20.149.794.461      instructions     #  0,34  insn per cycle

      17,378575869 seconds time elapsed
```

```
$ perf stat python -c "import BS; BS.run(4000000,100000000)"

      25.234,65 msec task-clock       #  1,000 CPUs utilized
 85.324.155.833      cycles           #  3,381 GHz
 21.440.142.300      instructions     #  0,25  insn per cycle

      25,243117117 seconds time elapsed
```

Explica brevemente la variación en el tiempo total de ejecución del programa al duplicar el valor de N = 1M a N = 2M y luego al duplicarlo otra vez a N= 4M.